COMPUTER VISION / AERIAL SURVEILLANCE

Drone Detection with YOLO11n

Static-image detection prototype for drone-related imagery.

DETECT | LOCATE | YOLO11N

Python · Google Colab · Ultralytics



Made with GAMMA

WHY IT MATTERS

Why automated drone detection matters



Manual image review

Manual image review is slow and inconsistent.



Object detection

Object detection identifies both the object class and its location.



Fine-tuning

This project explores fine-tuning a pretrained detector for drone-related imagery.

Made with GAMMA

PROJECT SCOPE

One end-to-end image workflow, from download to inference.

The prototype runs in Google Colab and covers dataset download, annotation parsing, YOLO conversion, training, evaluation, and inference.

01	02	03	04	05	06
Download	**Parse**	**Convert**	**Train**	**Evaluate**	**Infer**
Dataset	Annotations	YOLO format	YOLO11n	Metrics	Output

OUT OF SCOPE

Not a continuous video tracker, drone-control system, or deployed monitoring service.

Made with GAMMA

TECHNOLOGY STACK

Technology stack

Python + Google Colab

Execution and experimentation.

KaggleHub

Dataset download.

Ultralytics YOLO11n + PyTorch

Detection and training.

Pandas, Pillow, Matplotlib, PyYAML

Data handling, inspection, visualization, and configuration.

Made with GAMMA

DATASET PREPARATION

Raw files become labels the detector can learn from.

01	02	03	04	05
Discover	**Identify**	**Export**	**Convert**	**Organize**
Images and annotation / configuration files	COCO-style JSON with images, annotations, and categories	Ground-truth boxes to CSV	Valid boxes into normalized YOLO labels	Labels and images in split folders

INPUT	STRUCTURE	AUDIT TRAIL	MODEL READY	TRAINING
Raw dataset	COCO JSON	CSV annotations	YOLO labels	Split folders

Made with GAMMA

DATA CONTRACT

Every image needs a repeatable split and a precise label.

TRAIN	VALIDATION	TEST
≈70%	≈20%	≈10%

FIXED SEED · 42

Randomization is repeatable across runs.

YOLO LABEL FORMAT

class_id · normalized center coordinates · normalized width · normalized height

Actual image counts, class names, and annotation totals must be recorded after execution.

Made with GAMMA

MODEL TRAINING

Train a compact detector with a controlled configuration.

01	02	03	04	05
Initialize	**Configure**	**Optimize**	**Schedule**	**Save**
yolo11n.pt Pretrained model	30 max epochs 640 image size	Batch size 8 AdamW optimizer	Patience 8 Cosine learning rate	**best.pt** Best checkpoint

CONTROLLED EXPERIMENT

The configuration is explicit before performance is interpreted.

Made with GAMMA

PERFORMANCE MEASUREMENT

Metrics separate correct detections from complete coverage.

PRECISION

—

How many predicted objects are correct.

RECALL

—

How many annotated objects are detected.

MAP@0.5

—

Detection quality at a 0.50 overlap threshold.

MAP@0.5:0.95

—

Stricter localization quality across multiple thresholds.

RESULTS PENDING

Measured values are not available because the uploaded notebook contains no executed outputs.

Made with GAMMA

INFERENCE OUTPUT

Predictions become inspectable artifacts.

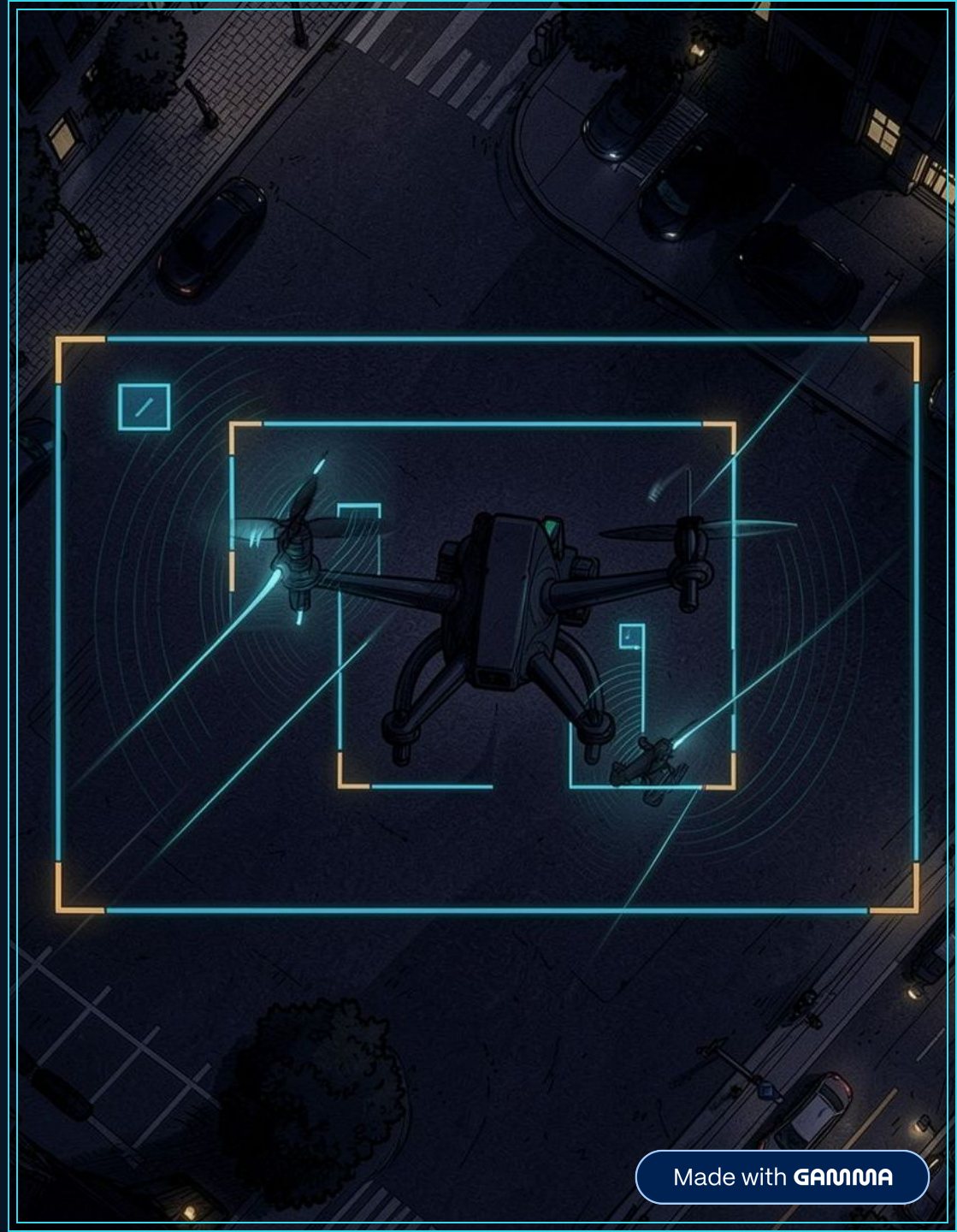
Upload an image and run the trained model at confidence threshold 0.25.

Annotated image — Predicted bounding boxes and class labels

drone_predictions.csv — Class, confidence, and box coordinates

THRESHOLD · 0.25

The annotated image and CSV make the result easy to inspect and reuse.



Made with GAMMA

VALIDATION GATE

Verify the pipeline before claiming success.



Data exists

Training and validation image counts are non-zero.



Labels resolve

Class IDs, YAML paths, and label coordinates are valid.



Samples align

Sample images match their generated labels.



Training completes

Best checkpoint exists and validation metrics are recorded.



Test split is held out

Evaluate it before drawing final conclusions.

Made with GAMMA